# Lecture Notes
# Observer Design Pattern

*Course: SWE 4104 Software Design Patterns*
*Topic 1: Case Study Anatomy of failure in the Observer Pattern*

## Structural Violations, Anti-Patterns, and Language Manifestations in Java

The primary behavioral intent of the Observer pattern is to establish a one-to-many dependency between objects so that when one object (the Subject) changes its internal state, all its dependents (the Observers) are notified and updated automatically.

Architecturally, this pattern aims to **achieve maximum decoupling**. The Subject must know absolutely nothing about the **concrete identities**, **memory layouts**, or **operational mechanics of the objects it notifies**. It should **recognize only a thin behavioral contract**. When developers **misapply object-oriented structural relationships—substituting Generalization for Realization**, or **Composition for Aggregation—the pattern collapses into an anti-pattern**. This case study **maps out these critical design failures, their programmatic impacts, and their programmatic implementations in Java**.

## 2. Structural Anti-Patterns Ranked by Severity

### Anti-Pattern 1: The Inheritance Trap (Generalization)

**The Structural Mistake:** Making a concrete Observer extend a concrete Subject class to gain access to data fields, or vice versa.
**Architectural Failure:** This violates the basic semantic meaning of Generalization. **Inheritance represents an immutable "is-a" relationship**. An Investor or a MobileApp display is fundamentally not a StockMarket.
**The Fragile Base Class Problem:** Because inheritance provides "**white-box**" reuse, the internal implementation details of the Subject leak directly into the Observer. If the internal state tracking or variable names of the Subject alter during optimization, the compiling pipeline of every observer crashes instantly. Furthermore, because Java enforces a single inheritance limit, using generalization here squanders the observer's single chance to extend a domain-appropriate parent class (cannot observe another entity i.e. Subject).

### Anti-Pattern 2: The Concrete-Concrete Association

**The Structural Mistake:** The Subject retains an explicit structural collection composed entirely of concrete observer types, **bypassing an abstraction boundary or interface**.
**Architectural Failure:** This directly violates the **Open/Closed Principle** (OCP). The Subject **is now open for modification** and **closed to extension**. Every single time a new type of device, logging monitor, or algorithmic trader needs to observe the system, the developer must open the Subject's core class file, inject a brand-new concrete list type,

update the subscription registry methods, and add an entirely new iteration loop within the notification system.

### Anti-Pattern 3: Lifecycle Entrapment (Composition)

**The Structural Mistake:** Storing the observer references inside a Composition link (filled diamond) instead of an Aggregation link (hollow diamond).
**Architectural Failure:** Composition implies existential lifecycle ownership. The parts are instantiated inside the constructor of the whole and cannot live outside it. When applied to the Observer pattern, this means the Subject dictates exactly when observers are created and destroyed. It completely prevents dynamic runtime registration. Objects cannot subscribe or unsubscribe based on runtime application context or user interaction; they are permanently shackled to the memory lifecycle of the Subject.

## 3. Anti-Pattern Implementations in Java

### Java Code Exhibit A: Generalization Anti-Pattern

```
class StockMarket {
   protected double currentPrice;
   public void updatePrice(double newPrice) {
      this.currentPrice = newPrice;
   }
}

// ANTI-PATTERN: Generalization introduces tight coupling and shatters encapsulation
public class ConcreteInvestor extends StockMarket {
   private String investorName;
   public ConcreteInvestor(String name) {
      this.investorName = name;
   }
   public void inspectMarketState() {
      System.out.println("Investor " + investorName + " reads price via inheritance: " +
currentPrice);
   }
}
```

### Java Code Exhibit B: Concrete Association Anti-Pattern

```
import java.util.ArrayList;
import java.util.List;

class EmailNotifier {
   public void sendEmail(double price) { System.out.println("Email Sent: " + price); }
}

class MobileAppDisplay {
```

```
    public void pushNotification(double price) { System.out.println("Push Sent: " + price); }
}

// ANTI-PATTERN: Subject explicitly couples itself to every concrete consumer type
public class StockMarketSubject {
    private List<EmailNotifier> emailObservers = new ArrayList<>();
    private List<MobileAppDisplay> mobileObservers = new ArrayList<>();

    public void attachEmail(EmailNotifier en) { emailObservers.add(en); }
    public void attachMobile(MobileAppDisplay mad) { mobileObservers.add(mad); }

    public void notifyAllConsumers(double price) {
        for (EmailNotifier email : emailObservers) {
            email.sendEmail(price);
        }
        for (MobileAppDisplay app : mobileObservers) {
            app.pushNotification(price);
        }
    }
}
```

### Java Code Exhibit C: Composition Lifecycle Anti-Pattern

```
import java.util.ArrayList;
import java.util.List;

interface MarketListener {
    void handleUpdate(double price);
}

class StandardDisplay implements MarketListener {
    public void handleUpdate(double price) { System.out.println("Display: " + price);\ }
}

// ANTI-PATTERN: Observers are structurally bound via composition
public class HardcodedStockMarket {
    private List<MarketListener> listeners;

    public HardcodedStockMarket() {
        this.listeners = new ArrayList<>();
        // Existential entrapment: Observers are hardcoded inside the constructor
        this.listeners.add(new StandardDisplay());
    }
}
```

## 4. Architectural Remediation (The Clean Design)

To establish a fully decoupled, production-grade Observer architecture, three strict structural constraints must be coordinated together:

1. Realization (Interface Inheritance): All consumer components must implement an identical, single-method interface contract (Observer).
2. Aggregation (Weak Reference Ownership): The Subject maintains a single collection referencing the interface layer. These elements are injected into the array from the outside via explicit runtime subscription pathways.
3. Dependency (Transient Invalidation): During execution, the Subject loops through the abstract references, interacting via a short-term method signature boundary.

### Clean Java Architecture Implementation

```
import java.util.ArrayList;
import java.util.List;

// 1. REALIZATION CONTRACT
interface Observer {
   void update(double data);
}

class EmailClient implements Observer {
   public void update(double data) { System.out.println("Email updated to: " + data); }
}

class MobileClient implements Observer {
   public void update(double data) { System.out.println("Mobile push updated to: " + data); }
}

public class DecoupledSubject {
   // 2. AGGREGATION: Injected list of abstract interfaces
   private final List<Observer> observers = new ArrayList<>();

   public void registerObserver(Observer o) {
      if(o != null && !observers.contains(o)) observers.add(o);
   }

   public void unregisterObserver(Observer o) {
      observers.remove(o);
   }

   public void notifyObservers(double currentMetric) {
      // 3. DEPENDENCY: Loop executes over abstractions safely
      for (Observer observer : observers) {
```

```
            observer.update(currentMetric);
        }
    }
}
```